

Distributed Systems

DAT520 - Spring 2026

Introduction to Gorums

Prof. Hein Meling



- Introduction
- Background: Quorums and Applications
- Gorums' Abstractions
- Several Case Studies and Some Experimental Evaluation
- Conclusions and Feedback

- Computers and networks are unreliable and fail
- **Want:** Fault Tolerance and Availability
- **Solution:** Replicate state across a set of nodes
- **Challenge:** *(strong) consistency*
- Requires complex protocols

Simplify Design and Implementation of Fault-tolerant Quorum-based Protocols

“A quorum is the minimum number of members of a deliberative assembly necessary to conduct the business of that group.” (Wikipedia)

Quorum Systems

- Collection of subsets of replicas, where every pair of subsets intersect.

- **Example:** Set of majorities

- Set of replicas:

$$\{a, b, c\}$$

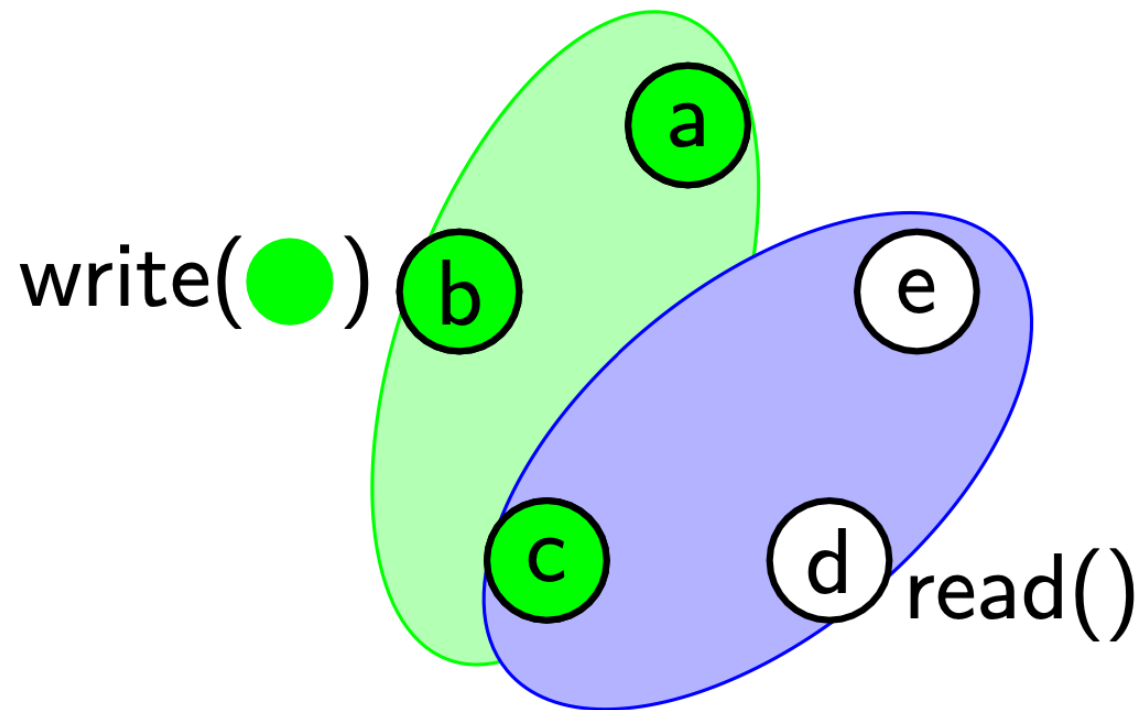
- Subsets that intersect:

$$\{\{a, b\} \cap \{a, c\} = a\},$$

$$\{\{a, b\} \cap \{b, c\} = b\},$$

$$\{\{a, c\} \cap \{b, c\} = c\}$$

Majority Quorum System



Other Types of Quorum Systems

- Read/write quorums
- Weighted quorums
- Grid quorums
- Byzantine quorums

Where are Quorum Systems Used?

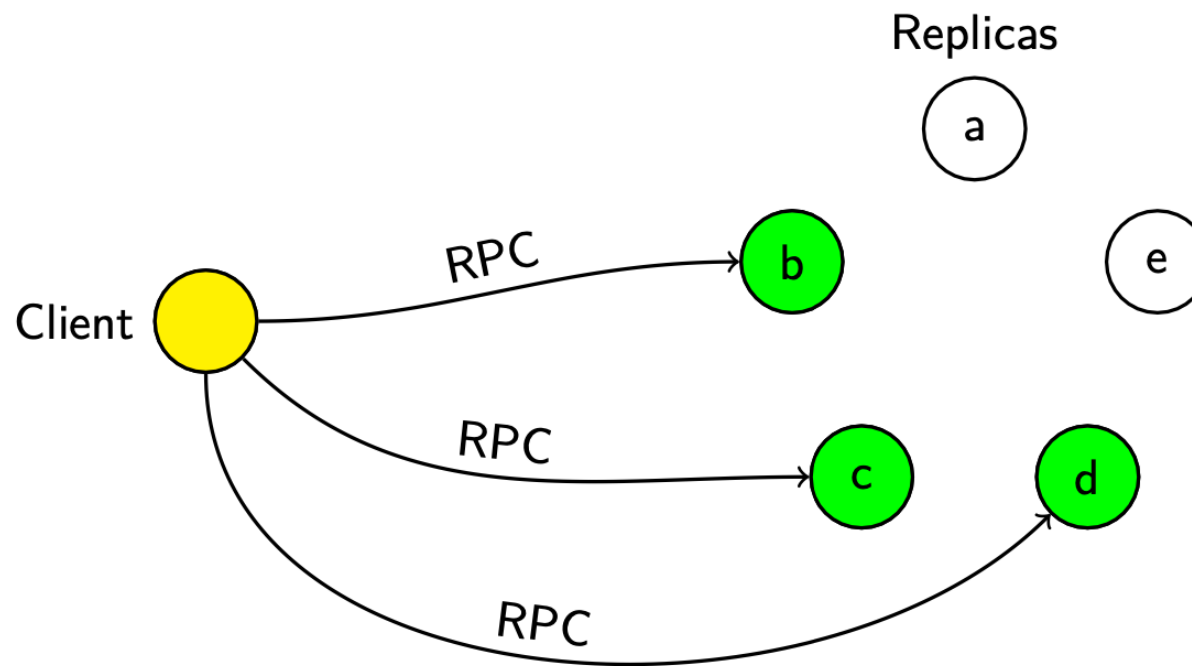
- Consensus and replicated state machines (RSMs)
- Distributed atomic storage
- Group communication systems
- Replicated databases
- Blockchains

How to build a Quorum System?

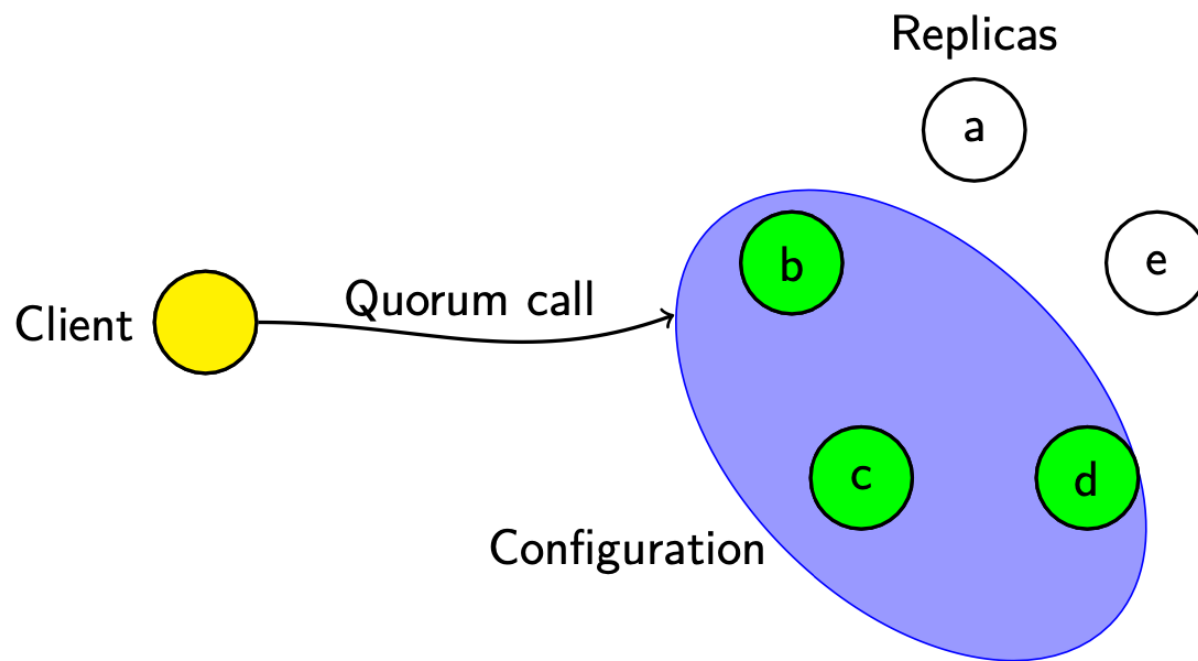
Invoke a Quorum of Replicas

- Access state stored at each replica
- To contact a quorum:
 - Must collect and associate replies from individual replicas
 - Not difficult in general, but adds complexity

Invoke a Quorum of Replicas: Using RPC



Invoke a Quorum of Replicas: Using Quorum Call



A Single-server Read/Write Storage

```
service Storage {  
    rpc Read(ReadRequest) returns (State) {  
  
    }  
    rpc Write(State) returns (WriteReply) {  
  
    }  
}
```

A Quorum-based Read/Write Storage

```
service Storage {  
    rpc Read(ReadRequest) returns (State) {  
        option (gorums.quorumcall) = true;  
    }  
    rpc Write(State) returns (WriteReply) {  
        option (gorums.quorumcall) = true;  
    }  
}
```

The Preamble (for Completeness)

```
syntax = "proto3";
```

```
package storage;
```

```
option go_package = "dat520/proto/lecture/storage";
```

```
import "gorums.proto";
```


Protobuf Messages (for Completeness)

```
message State {  
    string Value      = 1;  
    int64 Timestamp = 2;  
}  
  
message WriteReply {  
    bool New = 1;  
}  
  
message ReadRequest {}
```

Protobuf Messages (for Completeness)

```
$ go list -m -f {{.Dir}} github.com/relab/gorums
```

```
$ protoc -I="{{ proto_include }}" \  
    --go_out=paths=source_relative:. \  
    --go_opt=default_api_level=API_OPEN \  
    --gorums_out=paths=source_relative:. \  
    lecture/storage/storage.proto
```

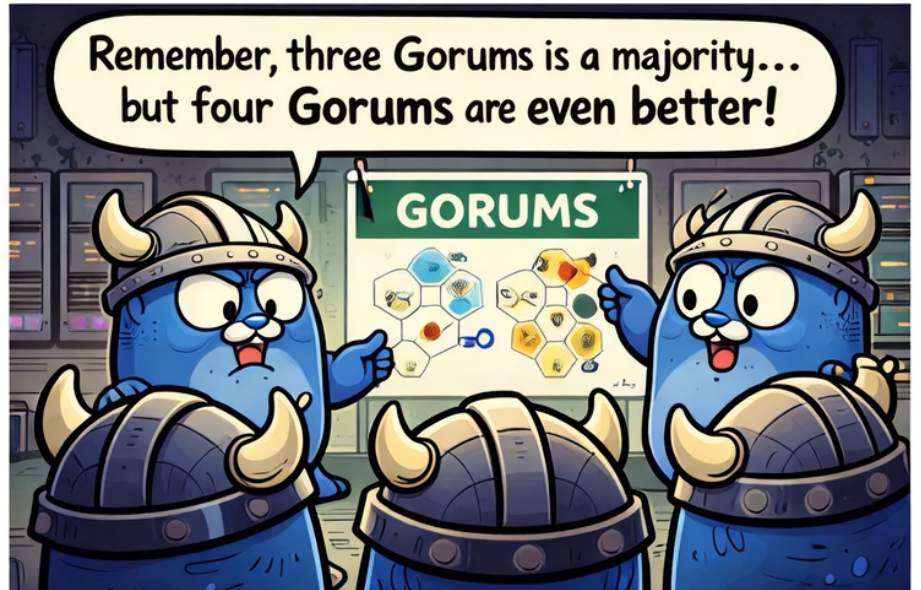
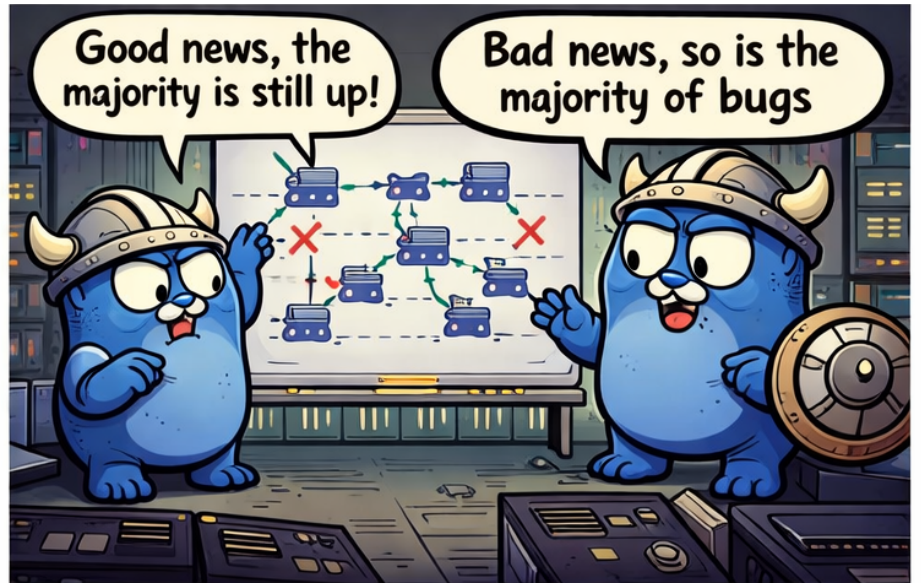
```
func Read(ctx *gorums.ConfigContext,  
         in *ReadRequest,  
         opts ...gorums.CallOption)  
    *gorums.Responses[*State]
```

```
func Write(ctx *gorums.ConfigContext,  
          in *State,  
          opts ...gorums.CallOption)  
    *gorums.Responses[*WriteReply]
```

```
type StorageServer interface {  
    Read(ctx gorums.ServerCtx, request *ReadRequest)  
        (response *State, err error)  
    Write(ctx gorums.ServerCtx, request *State)  
        (response *WriteReply, err error)  
}
```

Joke Time!





Douglas Adams

- In the story, a supercomputer called **Deep Thought** is asked to compute “*the Answer to the Ultimate Question of Life, the Universe, and Everything.*”
- After 7.5 million years of calculation, it delivers the answer:

42

- The joke is that **no one knows what the actual question is**, so the answer is useless on its own.

Douglas Adams

- Did 42 have a hidden meaning?
- Douglas Adams repeatedly said no. He chose 42 because it sounded ordinary and funny—precisely to parody the human desire for deep, universal explanations. Any attempts to find mathematical, religious, or scientific significance are part of the joke.

Douglas Adams

- **Why people still use** it today
- 42 has become a cultural shorthand for:
 - Absurdity in the face of over-serious questions
 - Playful skepticism about “ultimate explanations”
 - Geek and hacker humor (especially in CS and engineering)
- In short: **42 is funny because it pretends to be profound—but isn't.**

Gorums Concepts

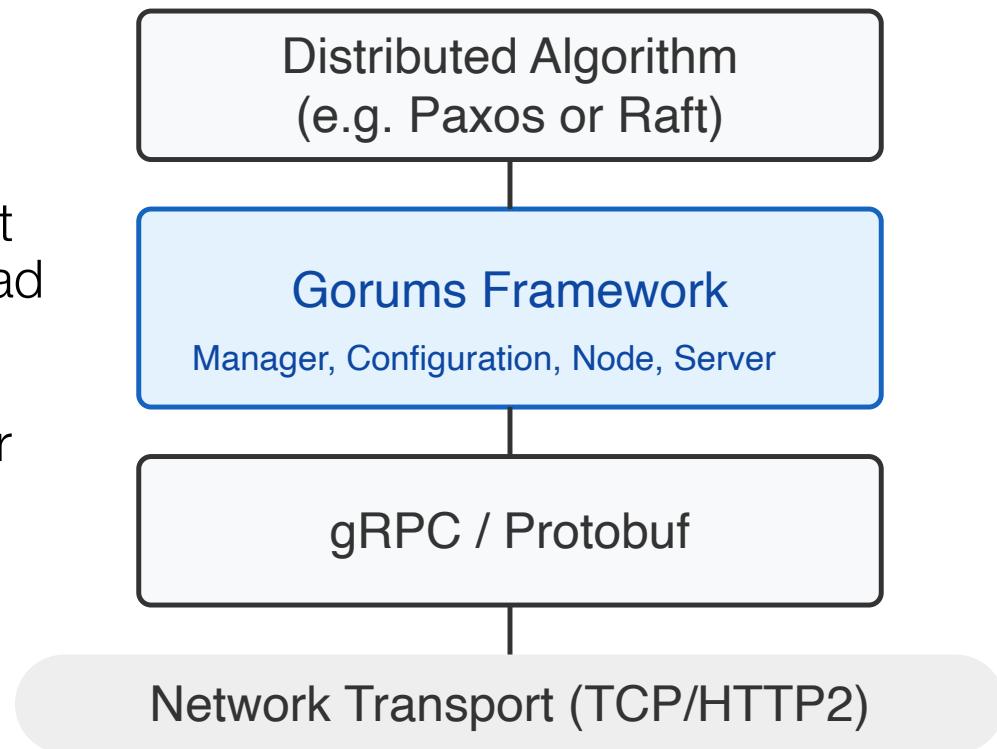
A framework for building fault-tolerant distributed systems in Go

- Separation of concerns
 - Selecting processes
 - Processing replies
- Built on top of gRPC
- Key Feature: **Flexible Quorum Calls**

Key Concepts

- **Manager:** Manages the connection pool to all nodes
- **Configuration:** A specific view or subset of nodes (e.g., "All Nodes", "Quorum Read Set")
- **Node:** Represents a single remote server
- **Server:** The actual implementation logic running on each machine

Gorums Abstraction Stack



Configuration

- A static set of N gRPC server replicas

```
type Configuration []*Node
```

Node

- A single gRPC server

```
type Node struct {  
    id    uint32  
    addr  string  
    // internal details  
}
```

Calltypes

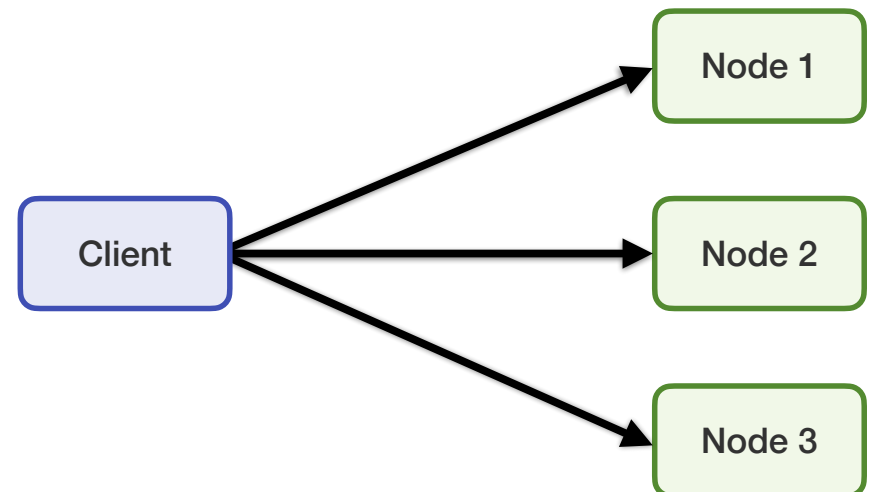
One-way Interaction

- Can return a send error, but not a server-side error
- Can ignore the error via `CallOption`



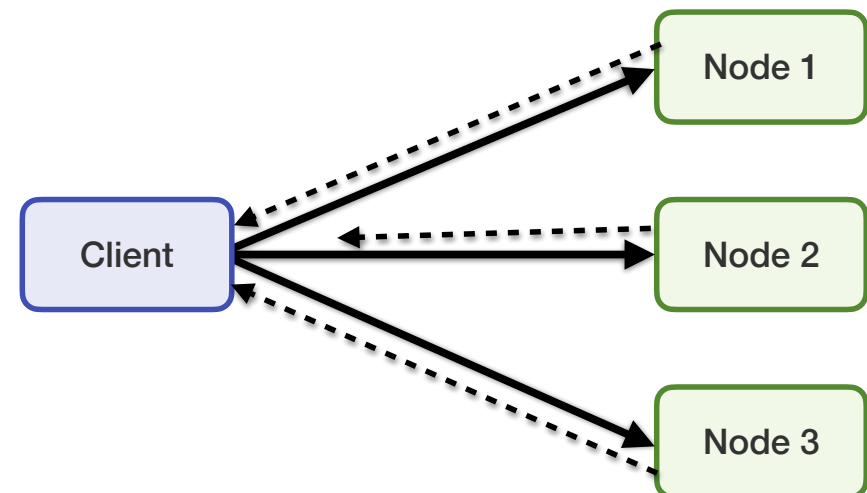
One-way Interaction

- Can return a send error, but not a server-side error
- Can ignore the error via `CallOption`
- Does not use the underlying multicast communication



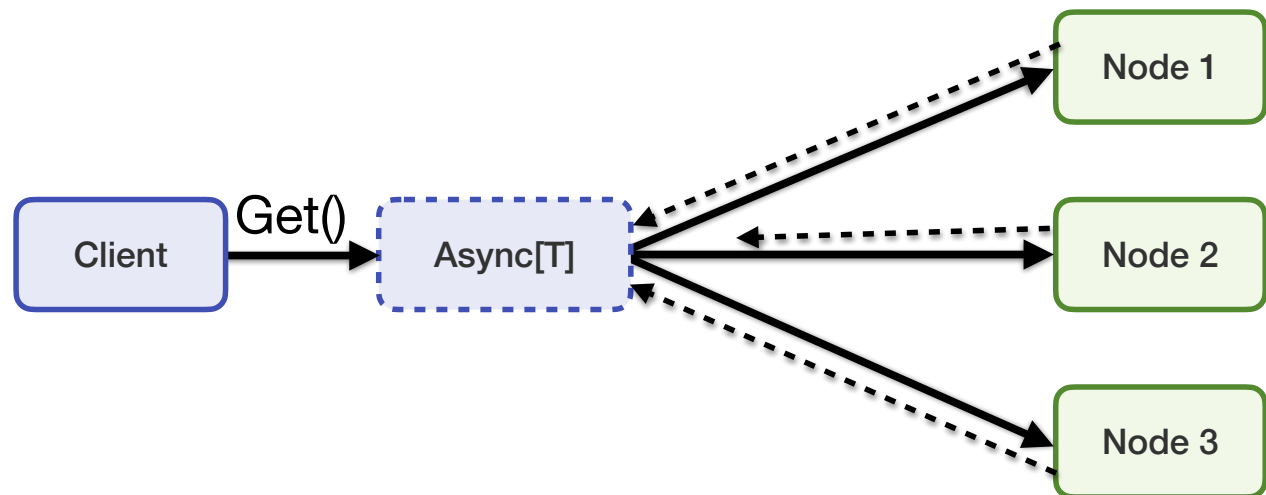
Two-way Interaction

- Does not use the underlying multicast communication
- Response processing
 - Library functions
 - Custom functions



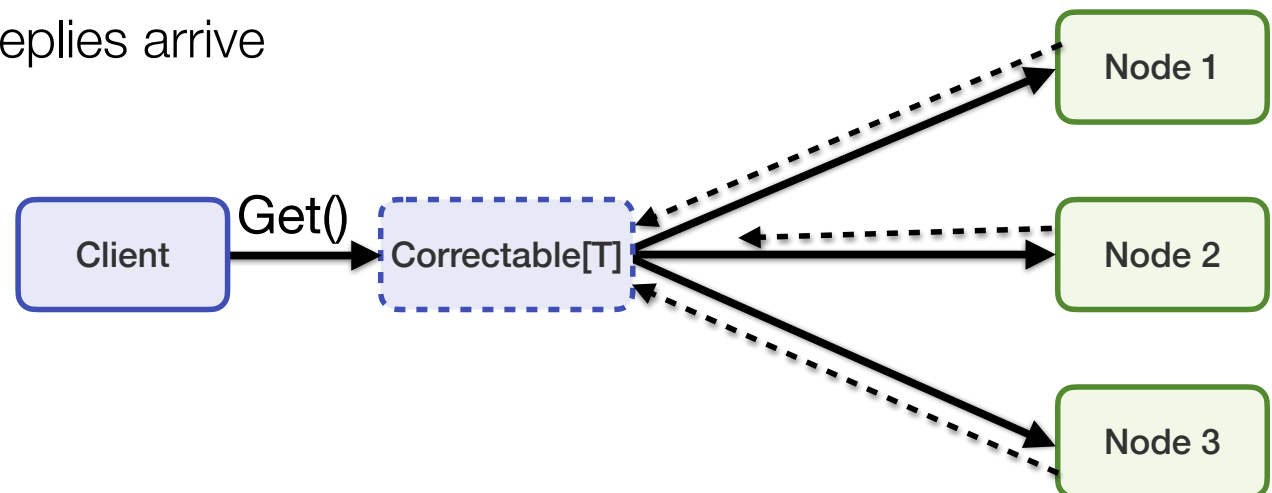
Two-way Interaction: Asynchronous Variant

- Delayed Response Processing



Two-way Interaction: Correctable Variant

- Delayed Response Processing
- Value is updated as more replies arrive
- Also supports streaming

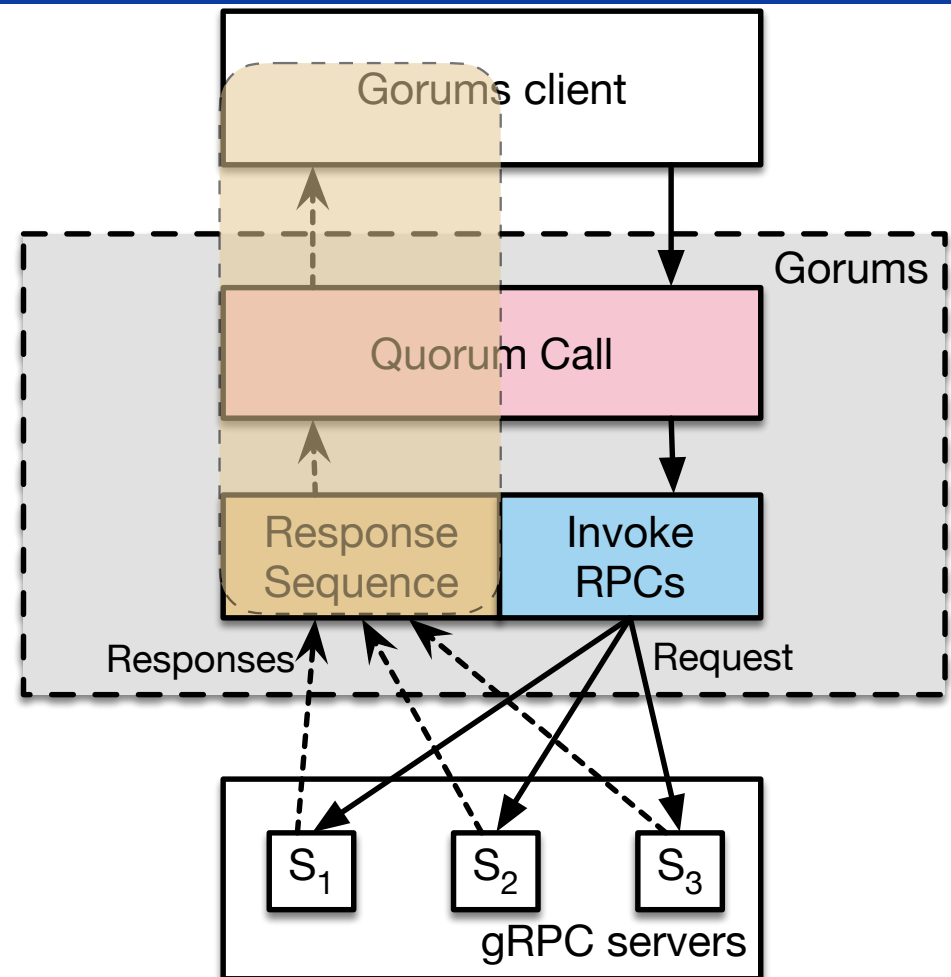


Gorums Abstractions

- Quorum Call
- Response Processing Functions

Quorum Call

- Invoke quorum call on a configuration
- Returns a response sequence
- Lazy invocation
- Trigger sending when using the response sequence



Quorum Call

- Invoke quorum call on a configuration
- Returns a response sequence

```
import clt "dat520/proto/lecture/storage"  
...  
var config gorums.Configuration  
ctx ...  
responses := clt.Read(config.Context(ctx), readReq)
```

Gorums Provided: Terminal Methods

- Blocks until the terminal method returns or timeout
- Returns a single response or an error

```
responses := clt.Read(config.Context(ctx), readReq)
resp, err := responses.First()
resp, err := responses.Majority()
resp, err := responses.All()
resp, err := responses.Threshold(k)
```

Terminal Method: Simple Majority

```
func (r *Responses[Resp]) Majority() (Resp, error) {  
    quorumSize := r.size/2 + 1  
    return r.Threshold(quorumSize)  
}
```

Gorums Provided: Terminal Methods

- Blocks until the terminal method returns or timeout
- Returns a single response or an error

```
ctx := cfg.Context(ctx) // ctx with timeout  
// Blocks until majority replies or timeout  
reply, err := clt.ReadQuorumCall(ctx, req).Majority()
```

Custom Aggregation

- Examine the content of individual responses
- Responses is an iterator that yields `NodeResponse[T]` values

Custom Aggregation

- Examine the content of individual responses

```
type ResponseSeq[T msg] iter.Seq[NodeResponse[T]]

type NodeResponse[T any] struct {
    NodeID uint32
    Value  T
    Err    error
}
```

Custom Aggregation

- Examine the content of individual responses
- Ignore errors if we don't care about individual server errors

```
responses := clt.Read(config.Context(ctx), readReq)
var sum int32
for resp := range responses.Seq().IgnoreErrors() {
    sum += resp.Value.GetValue()
}
```

Response Processing: Quorum Logic

- A **quorum call** needs to determine
 - if a quorum of responses has been received
 - what kind of response to return
- Implement a custom response processing function
 - quorum logic can involve more advanced processing

Custom Aggregation Function

```
// MaxTimestamp returns the state with the highest timestamp.  
func MaxTimestamp(responses *gorums.Responses[*State]) (*State, error) {  
    var max *State  
    // Iterate over responses as they arrive  
    for resp := range responses.Seq() {  
        if resp.Err != nil {  
            continue  
        }  
        if max == nil || resp.Value.Timestamp > max.Timestamp {  
            max = resp.Value  
        }  
    }  
    return max, nil  
}
```

Questions?